第15天 · 用依赖图组织多步骤任务

先知道哪些步骤必须等待，再讨论自动执行。

标准路线：60分钟；忙碌日：20分钟；扩展练习另计。

今天的目标

- 把任务写成步骤和依赖关系。

- 阅读拓扑排序给出的可行顺序。

- 通过真实循环依赖观察计划验证失败。

开始之前

- 完成第14课，或能在practice目录运行日课命令。

- 只使用教学材料；修改配置前保留原值，实验结束后恢复或记下修改。

一小时安排

- 15分钟 · 理解概念：阅读四节讲解，记下今天最关键的一个区别；外部资料按指定小节选读。

- 30分钟 · 操作与修改：先完成四步带练，再改变一个参数，保留修改前后的结果。

- 10分钟 · 验收与自测：对照三项验收，完成两道自测并阅读解析。

- 5分钟 · 记录与收尾：写下结果、未解决的问题和下次打开项目后的第一步。

01 · 理解概念

计划是带前提的动作集合

一个证据报告可以分成retrieve、analyze、draft与review。分析需要先拿到材料，草稿需要分析结果，审阅需要草稿。这些前提比单纯列出四个动词更重要，因为它们决定某一步是否已经具备输入。

今天用固定教学图演示依赖检查，不调用模型生成计划，也不真正运行四个科研节点。程序会检查图并给出可行顺序；接下来几课才逐步讨论执行失败、重试、批准和状态管理。

有向无环图表达可以完成的依赖

图的节点是任务，依赖表示执行某节点之前必须完成什么。本课graph用“节点→其前置节点列表”的JSON表示，例如analyze:[retrieve]表示分析依赖检索。不要把这个存储方向误读成分析要先于检索。

当依赖中没有环，可以按前提依次完成。若检索又等待最后的审阅，审阅等待草稿，草稿等待分析，分析等待检索，就没有任何步骤能够先开始。实际程序应拒绝这样的计划，而不是无限等待。

拓扑排序给顺序，不证明方法正确

拓扑排序寻找满足全部依赖的执行次序。当前实现会拒绝未知依赖和循环；无依赖的节点可以先安排。对于存在独立分支的图，可能有多个合理顺序，顺序不同不一定代表错误。

即使图有效，仍要检查每一步的方法、输入、权限与验收条件。例如“根据相关性宣布因果”的节点可以排列得很整齐，却没有科学依据。结构正确只是可执行计划的一项条件。

给每个节点写出完成标准

retrieve完成的标准可以是留下可定位来源；analyze完成的标准可以是记录支持与不支持的结论；draft需要引用与局限；review需要记录哪些项已核对。完成标准越具体，程序越容易判断是否可以进入下一步。

真实任务中还会有等待用户批准、可重试失败、预算不足与取消。这些状态不宜硬塞进“成功/失败”两个词。今天先把依赖讲清楚，之后再给节点增加适合的执行控制。

02 · 一步一步做

命令在解压后的 practice 文件夹执行。Windows可使用 .\.venv\Scripts\python.exe；macOS/Linux可使用 .venv/bin/python 替代 python。

1. 运行无环图

为什么做：从一个有效计划建立基线。

保持day15.force_cycle为false，执行本课。

```
python run.py day 15
```

预期结果：valid为true，execution_order满足retrieve、analyze、draft、review的依赖。

2. 逐项核对依赖方向

为什么做：确保读懂图的实际表示。

打开graph，对每个节点检查其前置节点是否出现在execution_order更前面。

```
python -m json.tool outputs/day15.json
```

预期结果：能用自己的话解释analyze依赖retrieve，而不是反过来。

3. 制造一个真正的环

为什么做：验证无效计划被拒绝。

只将day15.force_cycle改为true，再运行；程序会让retrieve反过来依赖review。

```
python run.py day 15
```

预期结果：valid为false、execution_order为空，并出现循环依赖错误说明。

4. 恢复并写节点验收

为什么做：把图结构与任务标准对应。

恢复force_cycle=false并重跑；为四个节点各写一个可检查的完成条件。

```
python run.py day 15
```

预期结果：图重新有效；笔记说明今天验证的是依赖结构，尚未执行这些专业任务。

改一改 · 一条反向依赖为什么让整个计划停住?

- 用force_cycle=false保存graph和execution_order。

- 仅将force_cycle设为true，比较graph中retrieve的依赖变化与error。

- 用一句话解释为什么任何节点都无法首先执行，然后恢复false。

观察什么：增加retrieve依赖review后形成闭环，拓扑排序拒绝计划并返回明确错误。

机制解释：依赖检查能在实际工具执行之前发现无法调度的计划，避免把逻辑问题推迟到运行中。

常见问题

把依赖表当成执行顺序

原因：JSON对象中的节点排列与前置依赖是两回事。

处理：查看execution_order，再逐项验证每个依赖是否在前。

计划有环却想靠多重试解决

原因：不存在满足全部前提的开始节点，重复运行不会消除逻辑矛盾。

处理：修改错误依赖或拆出阶段，恢复force_cycle=false验证；不要通过无限等待绕过。

03 • 验收

第15课运行结果、一次参数对照，以及自己能解释的学习笔记。

成果文件：outputs/day15.json

☐ 能正确解释graph中的前置依赖方向。

☐ 运行过正常图与force_cycle=true的实际失败对照。

☐ 为每个节点写出完成标准，并说明有效计划不等于科学方法有效。

自测与解析

1. analyze:[retrieve]在本课中表示什么?

- A. 先分析再检索

- B. 分析必须等待检索完成

- C. 二者必须同时执行

- D. 二者没有关系

答案：B。 列表记录该节点的前置依赖，因此retrieve应在analyze之前。

2. 拓扑排序成功能够说明什么?

- A. 研究设计已经正确

- B. 所有工具都执行成功

- C. 依赖结构允许至少一种执行顺序

- D. 无需人工复核

答案：C。 拓扑排序验证依赖与顺序，不能代替节点实际执行、方法审查或结果核验。

小词典

- DAG：有向无环图，用于表示没有循环等待的依赖。

- 拓扑排序：按前置依赖安排节点的可行顺序。

- 循环依赖：多个节点互相等待，形成无法首先开始的闭环。

04 · 我的记录

- 自己的任务中，哪一步最容易在输入还没准备好时就开始?

- 如果一个步骤失败，哪些下游任务需要等待，哪些仍能继续?

忙碌日：20分钟

- 用3分钟读“计划是带前提的动作集合”，写下一个关键区别。

- 用12分钟运行 python run.py day 15，打开 outputs/day15.json 并核对第一项预期结果。

- 用5分钟完成第一道自测并写一句解释；完整参数实验和其余验收留待下次。

留下本课运行结果与一句自己的解释；精简完成单独记录，不等于完整课程已经验收。

有余力再做：20分钟选学 · 把自己的小任务画成依赖图

- 选择一个只需要四至六步的工作。

- 写出每一步的输入、交付物与前置步骤。

- 找出可以独立开展的节点，并标出需要人工批准的动作。

参考资料与阅读范围

BioAgentLab：规划与 ReAct：关注行动、观察、停止条件与规划依赖，不把示意流程当成可靠性证明。

https://github.com/zbyloveyj/BioAgentLab/blob/main/book/zh/07_planning_and_react.md

Anthropic：Building effective agents：阅读 workflows 与 agents 的区别，以及从简单结构开始的讨论。

https://www.anthropic.com/research/building-effective-agents

下一步

下一课处理临时失败、重试次数与超时，让可行计划能够有边界地执行。